

BOOK STORE (BookNest)

Full FSD Project Documentation

Team ID: SWTID-2026-4823

Project: MERN Stack Online Bookstore Platform

Compilation Date: July 10, 2026

Phase 1: Brainstorming & Idea Generation

Customer Problem Statement Template

A well-articulated customer problem statement allows the team to find the ideal solution for the challenges your customers face. Empathizing with users helps understand how they perceive the bookstore service.

Problem Statement (PS-1): Customer / Reader Perspective

Element	Customer Input
I am...	An avid book reader or student looking for specific, curated, or academic textbooks and local literature.
I'm trying to...	Easily search, discover, and purchase books online, and support local independent booksellers.
But...	Major online marketplaces are cluttered, lack niche search filters, charge heavy shipping fees, and do not integrate regional local bookstore inventories.
Because...	There is no dedicated, specialized platform that digitizes and aggregates local bookstores' collections and inventory.
Which makes me feel...	Frustrated, disconnected from the local literary community, and forced to pay premium prices on corporate sites.

Problem Statement (PS-2): Seller / Bookstore Perspective

Element	Seller Input
I am...	An independent bookstore owner, regional publisher, or individual selling pre-owned books.
I'm trying to...	Publish, list, and sell my physical book stock online to reach a wider base of digital customers.
But...	Building an independent e-commerce store is technically complex and expensive, while listing on giant marketplaces hides my shop, charges high commissions, and lacks direct inventory controls.
Because...	Current retail marketplace systems are optimized for generic high-volume commodities and ignore specialized book data (genres, authors, ISBNs, book conditions).
Which makes me feel...	Marginalized, financially strained, and unable to compete with major chain retailers in the digital space.

Phase 1: Brainstorming - Idea Generation Details

Brainstorming & Idea Prioritization Template

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process. Prioritizing volume over value, out-of-the-box ideas are welcome and built upon, helping developers discover rich solutions.

Step-1: Select the Problem Statement

Selected Core Problem Statement: How can we build an intuitive, low-commission digital bookstore platform (BookNest) that connects local/independent book sellers with readers, enabling smart book discovery and seamless checkout?

Step-2: Brainstorm, Idea Listing and Grouping

Idea Category	Brainstormed Features / Ideas
Book Discovery & UX	1. Search autocomplete by title, author, and tags. 2. Multi-genre browsing with sidebar filters (price, rating, category). 3. Detailed book pages showing cover art, condition (New/Used), and seller info. 4. Wishlist and shopping cart for saving and grouping purchases.
Seller Enablement	1. Standardized book listing form (ISBN, title, author, description, price, stock). 2. Local filesystem uploads using Multer for book cover images. 3. Seller Order Management dashboard showing order logs and shipping status.
Platform Governance	1. Secure login and signup for Customers, Sellers, and Admins via JWT. 2. Admin portal to monitor total sales, view registered users, and delete policy-violating listings.

Step-3: Idea Prioritization

Category	Features	Priority / Decision
High Impact, Low Effort (Quick Wins)	User registration/login API, basic book searching, wishlist, shopping cart.	Immediate Implementation (Sprint 1 & 2)
High Impact, High Effort (Strategic)	Seller catalog uploads, image filesystem integration, simulated order checkout.	Plan & Develop (Sprint 2 & 3)
Low Impact, Low Effort (Fill-ins)	Book ratings and comments, seller profile personalization.	Nice to Have (Post-Release)
Low Impact, High Effort (Thankless)	Blockchain-based DRM tracking, integrated shipping route optimization API.	Discard / Defer

Phase 1: User Empathy Mapping

Empathy Map Canvas

An empathy map is a simple, easy-to-digest visual that captures knowledge about a user's behaviors and attitudes. It helps the team understand the user's perspective, goals, and challenges.

User Persona: Sarah (Avid Reader & College Student)

Empathy Quadrant	Customer Thoughts & Behaviors
SAYS	"I want to support local bookstore owners but I can't find their collections online." "Textbooks are extremely expensive new; I prefer buying verified second-hand copies."
THINKS	"Will my financial data be secure on this platform?" "How long will shipping take?" "I hope the description of the book's condition (New/Used) is accurate."
DOES	Searches multiple websites to compare book prices. Maintains a list of wanted books in notepad/wishlists. Checks online reviews and book recommendations on social media groups.
FEELS	Overwhelmed by massive commercial marketplaces filled with irrelevant ads. Excited when finding a hard-to-locate book. Relieved when checkout is straightforward and fast.
PAINS	High textbook prices. Fragmented inventory across local stores making search tedious. Slow shipping and unclear book conditions.
GAINS	Centralized search of multiple local store inventories. Affordable pre-owned books. Secure, quick checkout interface.

Phase 2: Data Flow Diagrams & User Stories

Data Flow Diagrams (DFD)

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. It shows how data enters and leaves the system, what changes the information, and where data is stored.

DFD Level 0 (Context Diagram) Description

The bookstore platform interacts with three external entities:

- 1. **Customer:** Sends signup, authentication, search queries, cart additions, and checkout shipping data. Receives search results, wishlist feedback, and order summary.
- 2. **Seller:** Sends listings creation forms and uploads cover art images via HTTP POST. Receives inventory status, order inbox logs, and dashboard metrics.
- 3. **Admin:** Sends account validation approvals and moderation deletion operations. Receives platform summary data, user lists, and logs.

DFD Level 1 (Process Breakdown) Description

- Process 1.0 (Auth Controller): Reads login/signup payloads, checks user credentials in MongoDB, and signs JWT session tokens.
- Process 2.0 (Book Catalog engine): Decodes search queries, applies filters (genre, min/max price), and queries MongoDB Books collection.
- Process 3.0 (Order Controller): Pulls cart items, decreases Book collection stock counters, and writes new order document to MongoDB Orders collection.
- Process 4.0 (Seller Dashboard engine): Processes seller books creation and routes files using Multer into the filesystem storage (/uploads).
- Process 5.0 (Admin Dashboard engine): Handles admin actions to query store summaries and delete book entries.

User Stories Matrix

Role	Epic	USN	User Story	Acceptance Criteria	Priority	Sprint
Customer	Registration	USN-1	As a new user, I can signup with my email/password, so that I can browse as an authenticated member.	Validates email, hashes password using bcrypt, sets default status 'approved'.	High	Sprint -1
Customer	Login	USN-2	As a user, I can login, so that my cart/wishlist syncs across sessions.	Returns JWT token, updates frontend AppContext state.	High	Sprint -1
Customer	Search	USN-3	As a shopper, I can search books and filter by genre/price, so I find specific books quickly.	Real-time query mapping, case-insensitive title matching, price range check.	High	Sprint -1
Customer	Cart Flow	USN-4	As a buyer, I can add books to wishlist/cart and edit quantities, so that I checkout multiple items.	Stores cart in frontend state, sums total price, manages stock limits.	Medium	Sprint -2
Customer	Checkout	USN-5	As a customer, I can checkout by inputting shipping info, so that I place a formal order.	Stores order details in Order schema, clears user cart, decrements stock.	High	Sprint -2

Seller	Inventory	USN-6	As a seller, I can upload books with cover art, so that they appear in search results.	Form validation, Multer files write to disk, maps sellerId to session.	High	Sprint -3
Admin	Governance	USN-7	As an admin, I can delete any book listing and block users, to maintain store policy.	Checks admin role on route, database removes document, pulls from wishlists.	Medium	Sprint -3

Phase 2: System Requirements Specification

Functional Requirements

Following are the functional requirements of the proposed online bookstore solution, mapped directly to our MERN stack controllers and database schemas.

FR No.	Requirement Epic	Description & Implementation Details
FR-1	User Management & Authentication	Support customer, seller, and admin signups and logins. Hash passwords using bcrypt and validate routes using authorization header JWT tokens. Default user status is 'approved'.
FR-2	Book Catalog Search & Filtering	Customers can retrieve books globally and filter by text search (title, author, tags) or select genres. Pricing filters and sort routines (price low-high, rating, newest) are handled on the server.
FR-3	Wishlist & Cart	Authenticated customers can add/remove books in their wishlist and cart. Item quantities can be adjusted, and total price is calculated client-side.
FR-4	Checkout & Order Tracking	Simulated checkout saves shipping details, computes shipping fees, creates an order document in the DB, and displays order tracking history under /orders/me.
FR-5	Seller Inventory Management	Sellers can add books, update prices/stocks, list items with cover images (uploaded via Multer), and view orders linked to their books in `/api/orders/inbox`.
FR-6	Admin Portal	Admins can access a unified dashboard (/api/admin/dashboard) displaying system summaries, delete listings, and approve/suspend seller accounts.

Non-functional Requirements

Following are the non-functional requirements targeting safety, speed, and platform scalability.

NFR No.	Non-Functional Requirement	Target & Implementation Plan
NFR-1	Usability	Responsive UI compatible with desktop, tablet, and mobile browsers, utilizing unified CSS stylesheets and intuitive navigation menus.
NFR-2	Security	Secure credentials hashing using bcrypt, validate token claims on backend API routes via jwt.verify, configure CORS origins strictly.
NFR-3	Reliability	Consistent backend error handling returning user-friendly messages (400/401/403/404/500). Database data constraints enforced via Mongoose schema validators.
NFR-4	Performance	Frontend initial load under 1.5 seconds via Vite code-splitting and asset bundling. Backend queries optimized by index mappings on MongoDB.
NFR-5	Availability	Continuous availability achieved by deploying stateless Express API on Render, static frontend assets on global CDNs, and database on MongoDB Atlas replica sets.
NFR-6	Scalability	3-tier separation (Client SPA, REST API, Atlas Cloud Database) ensures layers can scale independently under concurrent peak load.

Phase 2: Technical Architecture & Tech Stack

Technical Architecture

The platform is built using a MERN (MongoDB, Express, React, Node) stack. The design implements a 3-tier model separating the Client Interface, Backend REST API, and Database Layer. Communication is handled over HTTPS via JSON payloads.

Table-1: Components & Technologies

S.No	Component	Description	Technology Stack
1	User Interface	Single Page Application rendering dashboards for user, seller, and admin roles.	React.js (Vite), HTML5, CSS3, React Router DOM
2	Application Logic	RESTful API processing core operations, routing and validation middleware.	Node.js, Express.js
3	Authentication	Secure password hashing and stateless token-based session management.	JSON Web Tokens (JWT), bcryptjs
4	File Uploads	Handles book cover image multipart uploads, storing them in public folders.	Multer middleware, Local server filesystem
5	Database	Document-based relational data modeling and database queries.	MongoDB, Mongoose ODM
6	Cloud Hosting	Cloud platforms hosting frontend files, REST API, and data storage.	Vercel (Client), Render (Backend), MongoDB Atlas (Cloud Database)

Table-2: Application Characteristics

S.No	Characteristic	Description	Technology Used
1	Open-Source	The system uses free, modular, and modern open-source node packages.	React, Express, Mongoose, Vite
2	Security	Encrypt credentials, enforce auth middleware, configure CORS policies.	bcrypt (salt=10), JWT verification, CORS headers
3	Scalability	Stateless REST API allows horizontal scaling across backend nodes.	RESTful architecture, stateless JSON payloads
4	Availability	Multi-region replica sets prevent data loss during single node failures.	MongoDB Atlas Cluster replication
5	Performance	Quick rendering, caching, and minimum query load.	Vite bundling, Mongoose indexing on book fields (title, author, tags)

Phase 3: Backlog, Sprints & Project Planning

Product Backlog, Sprint Schedule, and Estimation

The product backlog is broken down into three sprints of two days each, moving from core setup to features release between July 5 and July 10, 2026.

Sprint	Epic	USN	User Story / Task Description	Points	Priority	Owner
Sprint-1	Registration & Auth	USN-1	Develop signup UI form and backend user schema validation.	3	High	Developer A
Sprint-1	Auth API	USN-2	Implement backend login endpoint returning JWT tokens.	2	High	Developer B
Sprint-1	Catalog	USN-3	Build homepage listing featured books and searching by text.	3	High	Developer A
Sprint-2	Cart System	USN-4	Create wishlist UI and cart item management logic.	3	High	Developer B
Sprint-2	Checkout	USN-5	Implement checkout simulated payment and order placement API.	3	High	Developer A
Sprint-2	Filtering	USN-6	Add advanced search sidebar filters (genre, price, sort).	2	Medium	Developer B
Sprint-3	Seller Portal	USN-7	Build seller dashboard and Multer cover image uploading API.	5	High	Developer A
Sprint-3	Admin Panel	USN-8	Create admin catalog review and user control page.	4	Medium	Developer B
Sprint-3	Deployment	USN-9	Execute testing validation, database seeding, and cloud deployment.	3	High	Both

Project Tracker, Velocity & Burndown Chart

Sprint	Planned Points	Duration	Start Date	End Date	Completed Points	Actual Release
Sprint-1	8	2 Days	July 5, 2026	July 6, 2026	8	July 6, 2026
Sprint-2	8	2 Days	July 7, 2026	July 8, 2026	8	July 8, 2026
Sprint-3	12	2 Days	July 9, 2026	July 10, 2026	12	July 10, 2026

Velocity & Burndown Chart Description

Average Velocity: The team completed 28 story points over 6 days, yielding an average velocity of 4.67 story points per day (or 9.33 story points per sprint).

Burndown Chart: The chart plots the remaining story points (y-axis) starting at 28 points on July 5 and sloping down to 0 on July 10, indicating on-time delivery across sprints.

Phase 4: Problem-Solution Fit Analysis

Problem – Solution Fit Canvas

The Problem-Solution Fit validates that the platform solves the specific customer/seller struggles identified in Phase 1.

Identified Pain Point	Proposed Solution Fit	Behavioral Validation
Identified Pain Point	Proposed Application Solution Fit	Behavioral Match & Validation
Avid readers cannot locate specialized books in local stores without travel.	Aggregated book catalog searchable globally by title, author, and genre.	Readers search a single site rather than browsing physical shelves in multiple shops.
New academic textbooks are too expensive for students.	A secure system allowing individual sellers to list second-hand copies.	Cost-sensitive students search the database and sort listings by lowest price.
Local bookstore owners lack coding knowledge to run e-commerce sites.	A pre-formatted backend dashboard with simple upload forms.	Sellers input text and upload photos without touching HTML or database scripts.
Sellers are deterred by massive online platform commissions.	Low platform commission structure (e.g. 5% transaction flat fee).	Sellers preserve profit margins, leading to lower prices and higher platform inventory.

Phase 4: Proposed Commercial Solution Profile

Proposed Solution Parameters

This document reviews key product parameters validating the commercial and social viability of the BookNest platform.

S.No	Parameter	Description
1	Problem Statement	Fragmented book inventory makes finding local/niche titles difficult for readers. Local booksellers lack a cheap, simple solution to showcase their catalogs online and compete with large e-retailers.
2	Idea / Solution Description	A MERN stack multi-role bookstore application allowing customers to search, filter, and buy books. Sellers easily upload inventory using direct forms. Admins manage site health and moderate listings.
3	Novelty / Uniqueness	Unified index bringing small shops and individual sellers into a single system; focus on book metadata (ISBN, genre, author); zero setup fee and low commission fees.
4	Social Impact	Saves small local businesses by digitizing their sales channels; provides students and readers with cheap second-hand book access, fostering local community support.
5	Business Model	Revenue is generated via a 5% commission on completed sales, featured premium catalog listings for high-volume sellers, and minor shipping coordination fees.
6	Scalability	React frontend is compiled into static assets and distributed via global CDNs (Vercel). Express backend handles stateless JSON payloads, allowing database horizontal scaling on MongoDB Atlas.

Phase 4: Detailed Solution Architecture Sequence

System Architecture

BookNest uses a stateless Express API backend and React frontend client communicating over HTTP REST protocols.

1. Data Layer: Database Schema Models

- **UserSchema:** Stores name, email, passwordHash, role (user/seller/admin), status (approved/pending/suspended), and wishlist (array of book IDs).
- **BookSchema:** Stores title, author, genre, price, stock, description, rating, featured, badge, cover colors object, tags, itemImage, and seller ID/name references.
- **OrderSchema:** Stores buyer details, expected delivery, shipping address sub-schema, items array (bookId, title, price, quantity, sellerId), subtotal, shippingFee, and total amount.

2. Routing & Controllers Layer: REST Endpoints

- Auth routes: /api/auth/register (POST), /api/auth/login (POST), /api/auth/me (GET)
- Catalog routes: /api/books (GET/POST), /api/books/:id (GET/PUT/DELETE)
- Wishlist routes: /api/wishlist (GET), /api/wishlist/:bookId (POST/DELETE)
- Orders routes: /api/orders (POST), /api/orders/me (GET), /api/orders/inbox (GET), /api/orders/:id/status (PATCH)
- Admin dashboard: /api/admin/dashboard (GET), /api/admin/users/:id/status (PATCH)

3. Checkout Request Data Flow

When a customer checks out their cart:

1. Client UI submits cart items and shipping details via POST request to `/api/orders` with JWT headers.
2. Backend `authMiddleware` validates JWT token using `jwt.verify` and decodes the buyer email/ID.
3. The database updates book stock counts using Mongoose `\$inc: { stock: -qty }` queries for each purchased book ID.
4. A new document is written to the Orders collection, and the customer's cart is set to empty in the database.
5. Express returns order details in a JSON payload; React displays the order summary screen.

Phase 5: User Acceptance Testing (UAT) and Bug Logs

Testing Overview

Parameter	Details
Project Name	BookStore (BookNest)
Version	v1.0.0
Testing Period	July 9, 2026 – July 10, 2026
Scope	User signup/login, Search/Filters catalog, Cart & Wishlist, Orders Checkout, Seller Dashboard, Admin moderations.
Environment	Local node host (http://localhost:5173 / http://localhost:4000), Local MongoDB Database.

UAT Test Cases

Test ID	Test Scenario	Steps	Expected Outcome	Status
TC-001	Customer Signup	1. Open signup. 2. Input email/pass. 3. Click Submit.	Account written in DB, redirects user to Login.	Pass
TC-002	Catalog Search	1. Enter 'Fiction' in search. 2. Click search button.	Book cards matching 'Fiction' are filtered and shown.	Pass
TC-003	Cart Checkout	1. Add book to cart. 2. Click Checkout. 3. Enter address.	Order written in Orders DB; stock decremented; cart cleared.	Pass
TC-004	Seller Add Book	1. Login as Seller. 2. Enter details and cover photo. 3. Submit.	Listing saved in DB; file uploaded via Multer to /uploads.	Pass
TC-005	Admin Moderation	1. Login as Admin. 2. Find book, click delete.	Book document is deleted from DB and removed from catalog.	Pass

Bug Tracking Log

Bug ID	Description	Steps to Reproduce	Severity	Status	Resolution
BG-001	Cover image upload failing on high res images (>5MB).	1. Attach 8MB JPEG cover in Seller Form. 2. Submit.	High	Closed	Added frontend dimensions validation and backend file limits check.
BG-002	Cart counter in navbar not updating instantly on additions.	1. Browse catalog. 2. Click 'Add to Cart'.	Low	Closed	Wrapped header component inside React Context Provider, syncing item counters instantly.

UAT Sign-off

Lead Tester: SWTID-2026-4823 Lead Tester (Signed on July 10, 2026)
Product Owner: Approved & Signed-off (Signed on July 10, 2026)